

Quando ordenar custa energia: o que 180 execuções revelam sobre Bubble, Insertion e Selection Sort

Um experimento com 10.000, 20.000 e 50.000 elementos

Estudantes: Arthur Carneiro Dantas Freire, Caio Victor de Sá Pessoa

Disciplina: Tópicos de Computação Avançada IV

Data: 18/09/2026

Em uma frase: a mudança de implementação pode alterar radicalmente o custo energético de um algoritmo, mas a complexidade assintótica e os detalhes do código precisam ser analisados juntos.

O experimento, em 30 segundos

Foram executadas 180 medições (6 versões × 3 tamanhos × 10 repetições), registrando consumo de energia, tempo de execução, tempo de usuário e tempo de sistema. A base utilizada contém três tamanhos de entrada: 10.000, 20.000 e 50.000 elementos. O resultado mais marcante aparece nas versões 2 de Insertion e Selection: com 50.000 elementos, a energia total caiu de 1.292,51J para 24,61J no Insertion e de 1.085,33J para 4,54J no Selection.

1. Por que medir energia em algoritmos de ordenação?

Ordenar dados parece uma operação puramente lógica, mas cada comparação, troca, atribuição e acesso à memória é executado em hardware que consome recursos. Em um cenário de uso isolado, uma diferença de alguns segundos pode parecer apenas uma questão de desempenho. Em aplicações que ordenam dados repetidamente, porém, a diferença se acumula: mais tempo de CPU significa mais tempo mantendo o processador ativo e, em geral, mais energia consumida.

O modelo de projeto consultado para este trabalho segue justamente essa lógica: comparar algoritmos não apenas pelo tempo teórico, mas também pelo comportamento observado no hardware. A página de referência do SustainableSE destaca que complexidade, tráfego de memória, cache e localidade podem influenciar o consumo energético na prática [1]. O guia metodológico do mesmo curso também recomenda repetir medições, controlar ruído e analisar a distribuição dos resultados em vez de olhar apenas para uma execução [2].

Neste trabalho, a energia foi tratada como a métrica principal, enquanto tempo de execução e tempo de CPU serviram para interpretar por que uma versão gastou mais ou menos. Essa escolha é coerente com a finalidade do experimento: não basta saber qual implementação termina primeiro; é necessário observar o custo computacional associado à execução.

2. Cenário computacional

Item	Informação a preencher
Processador	12ª Geração Intel® Core™ i7-12700 (12 Cores/25MB/20T/2.1GHz to 4.9GHz/65W)
Memória RAM	Memória de 16 GB DDR4 3200mhz
Sistema operacional	Linux
Python	3.14.7
Ferramenta de medição	perf stat 6.1.180
Kernel Linux	Debian 12
Condições do experimento	Outros programas desnecessários fechados durante as medições, alimentado pela tomada, ambiente interno com temperatura por volta de 24°C, realizados em um mesmo período para reduzir variações nas condições.

3. Como o experimento foi organizado

Cada um dos três métodos recebeu duas versões. Para cada versão foram testados vetores de 10.000, 20.000 e 50.000 elementos, com 10 execuções por combinação. Isso produz 30 medições por versão e 180 execuções no total. A repetição é importante porque o tempo e a energia variam um pouco de uma execução para outra devido ao sistema operacional, escalonamento do processador e outras atividades concorrentes.

Os dados da planilha registram cinco medidas principais: consumo de energia em joules, `user time`, `system time`, `duration time` e tempo total de CPU. Para a análise deste texto, usamos as médias das dez execuções de cada combinação. O boxplot também foi usado para não esconder a variação entre repetições.

Método	Versões	Tamanhos	Execuções
Bubble Sort	v1, v2	10k, 20k, 50k	10 por combinação
Insertion Sort	v1, v2	10k, 20k, 50k	10 por combinação
Selection Sort	v1, v2	10k, 20k, 50k	10 por combinação

Tabela 1 — Matriz experimental utilizada nas medições.

4. O que mudou entre as duas versões?

Bubble Sort. A segunda versão apresentou resultados ligeiramente piores na base medida. No tamanho de 50.000, por exemplo, a energia média passou de 2.579,110 J para 2.752,895 J, aumento de 6,74%, e o tempo de execução passou de 76,858 s para 78,121 s, aumento de 1,64%. Isso mostra que uma modificação aparentemente orientada à otimização não garante ganho em qualquer entrada. Quando os dados exigem muitas passagens, uma verificação adicional ou uma lógica de controle pode sobrecarregar o suficiente para anular o benefício.

Insertion Sort. Aqui a diferença foi muito maior. A versão 2 reduziu a energia em 96,47%, 97,62% e 98,10% para 10.000, 20.000 e 50.000 elementos, respectivamente. O padrão é compatível com uma implementação que reduz a quantidade de trocas ou movimentos de elementos, por exemplo, deslocando valores para abrir espaço e fazendo uma inserção ao final. A complexidade assintótica

pode continuar sendo $O(n^2)$, mas o número de operações efetivamente realizadas e o custo por operação caem de forma muito significativa.

Selection Sort. É a comparação mais importante do trabalho porque o enunciado exige uma segunda versão de complexidade $O(n \log n)$, baseada em uma variação do HeapSort. A versão 1 se comporta como Selection Sort quadrático; a versão 2 troca essa estratégia por uma estrutura de heap. Essa alteração modifica a ordem de crescimento do algoritmo e, nos resultados medidos, produz a maior diferença relativa em energia para o maior conjunto de entradas.

5. Resultados: quando o tamanho cresce, a diferença cresce junto

A Figura 1 é uma escala logarítmica no eixo vertical porque as versões não estão na mesma ordem de grandeza. Mesmo assim, o formato das curvas deixa o padrão evidente: Bubble e Selection v1 crescem muito mais rapidamente que Selection v2; Insertion v2 permanece em uma faixa muito menor que sua versão 1.

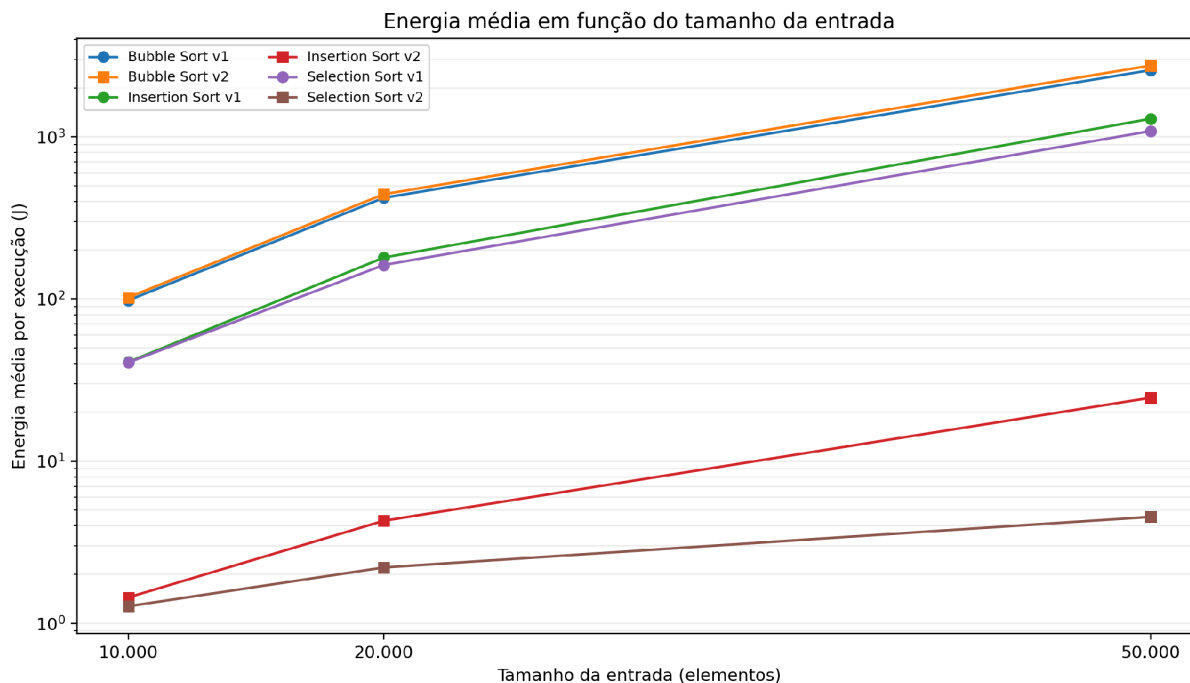


Figura 1 — Energia média por execução para os três tamanhos de entrada. Escala logarítmica no eixo vertical.

A Figura 2 mostra o mesmo fenômeno usando tempo de execução. A proximidade entre energia e tempo sugere que, neste experimento, a principal fonte da economia foi reduzir a duração do processamento. Isso é particularmente claro nas versões 2 de Insertion e Selection: quando o tempo cai de dezenas de segundos para menos de um segundo, o consumo energético também despenca.

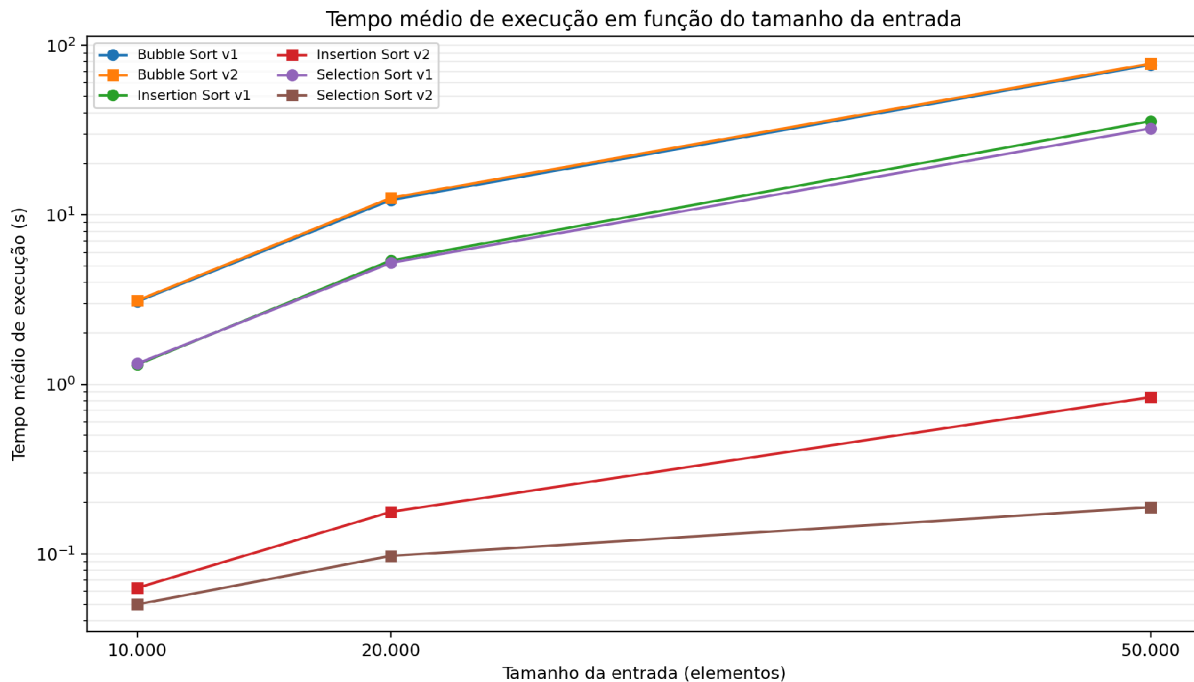


Figura 2 — Tempo médio de execução por combinação. Escala logarítmica para permitir comparação entre versões.

6. O caso mais expressivo: 50.000 elementos

Algoritmo	Versão	Energia média (J)	Tempo médio (s)	Variação de energia
Bubble Sort	v1	2579.110	76.858	—
Bubble Sort	v2	2752.895	78.121	+6,74%
Insertion Sort	v1	1292.513	35.762	—
Insertion Sort	v2	24.605	0.839	-98,10%
Selection Sort	v1	1085.327	32.368	—
Selection Sort	v2	4.536	0.188	-99,58%

Tabela 2 — Médias para 50.000 elementos. A variação compara cada v2 com a v1 do mesmo método.

O contraste é grande. No Selection, trocar o algoritmo quadrático pela versão baseada em HeapSort reduziu a energia média de aproximadamente 1,09 kJ para 4,54 J. No Insertion, a mudança de implementação reduziu de 1,29 kJ para 24,6 J. Já no Bubble, as duas versões continuam na mesma ordem de grandeza e a segunda foi um pouco mais cara no cenário testado.

7. A média conta a história toda? O boxplot ajuda a responder

Uma média é útil para comparar versões, mas pode esconder instabilidade. O boxplot da Figura 3 mostra as dez repetições de cada implementação com 50.000 elementos. As versões v1 de Bubble, Insertion e Selection estão em faixas de energia muito maiores; as v2 de Insertion e Selection aparecem próximas da base do gráfico. O formato também mostra que Insertion v1 tem dispersão bem maior que Selection v1, enquanto as versões rápidas têm pouca variação absoluta.

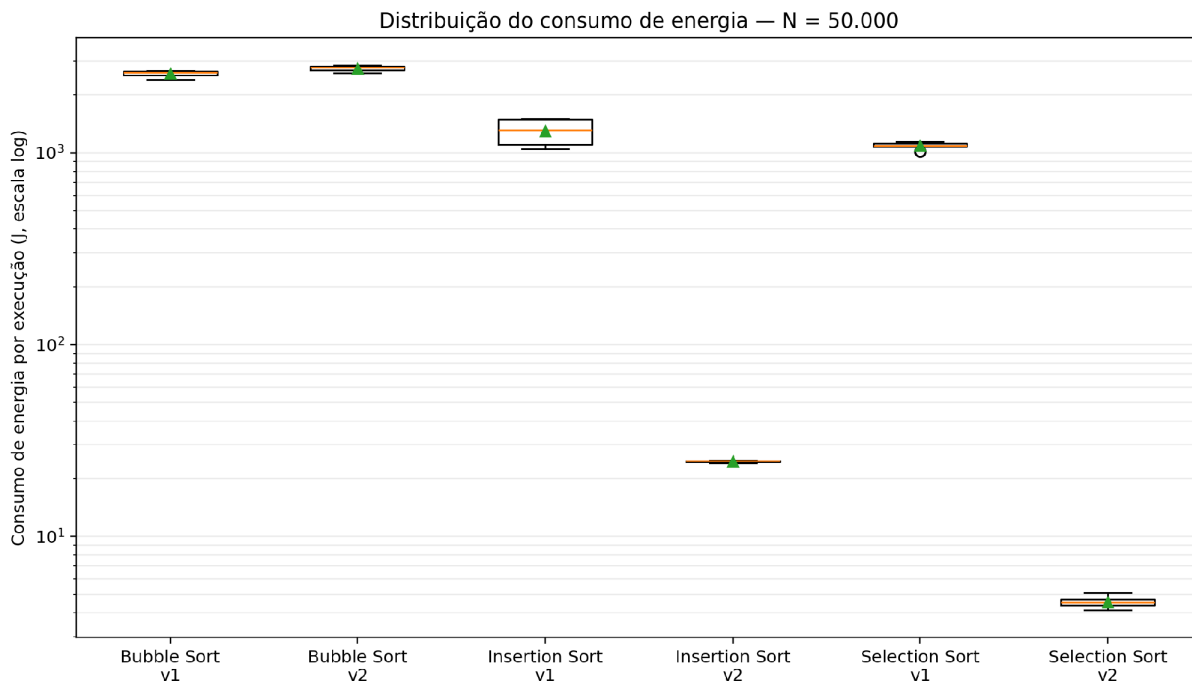


Figura 3 — Distribuição das dez medições de energia para N = 50.000. A escala vertical é logarítmica.

8. Redução de energia não é sinônimo de menor potência

É importante separar duas grandezas. Energia é a quantidade total consumida durante a execução, em joules. Potência é a taxa de consumo, em watts. Em termos simples, a potência média pode ser calculada por energia dividida pelo tempo em segundos. Assim, uma versão pode ter potência instantânea semelhante à de outra, mas ainda consumir muito menos energia porque termina mais cedo.

Isso ajuda a interpretar os resultados deste trabalho. A redução de energia das versões 2 de Insertion e Selection acompanha uma queda enorme do tempo de execução. Portanto, não é necessário supor que o processador ficou centenas de vezes menos potente; o que mudou principalmente foi quanto tempo a carga permaneceu ativa.

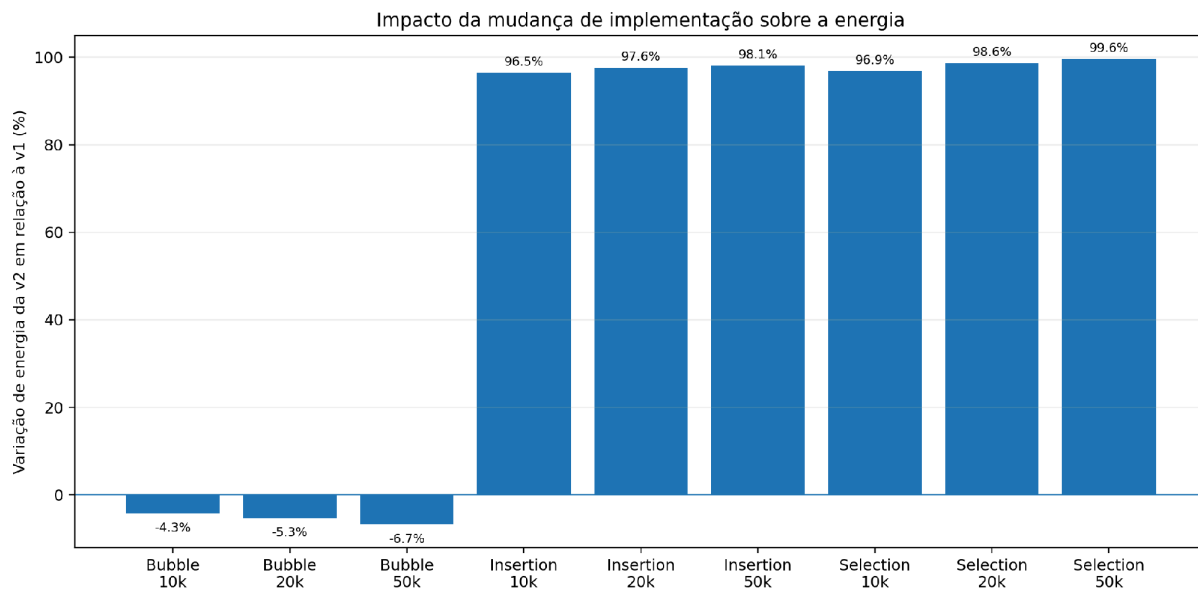


Figura 4 — Variação de energia da versão 2 em relação à versão 1. Valores negativos significam aumento de energia.

9. Por que uma mudança algorítmica pode economizar tanta energia?

A resposta passa por três camadas. A primeira é a complexidade assintótica. Um algoritmo $O(n^2)$ tende a executar uma quantidade de trabalho que cresce quadraticamente com o número de elementos. Quando o tamanho dobra, o esforço pode crescer aproximadamente quatro vezes. Os dados do experimento mostram exatamente esse comportamento aproximado nas versões quadráticas: Bubble v1 vai de 98,088 J para 420,694 J entre 10.000 e 20.000 elementos, enquanto Selection v1 vai de 40,565 J para 162,052 J.

A segunda camada é o número constante de operações. Mesmo quando duas implementações possuem a mesma classe $O(n^2)$, elas podem executar números muito diferentes de comparações, trocas, cópias e acessos à memória. É esse ponto que ajuda a explicar a diferença gigantesca no Insertion: a implementação v2 provavelmente diminui o trabalho efetivo por elemento. O resultado empírico confirma que a complexidade sozinha não determina o consumo observado.

A terceira camada é o comportamento do hardware. Acesso à memória, localidade de cache e padrões de execução também afetam o custo real. A página de referência do projeto do curso destaca justamente que dois algoritmos com complexidade semelhante podem apresentar consumos diferentes por causa do tráfego de memória e do cache [1]. Em outras palavras: Big O descreve o crescimento; não descreve sozinho tudo o que acontece dentro do processador.

10. O que os números dizem sobre cada método

Bubble Sort: as duas versões mantiveram o mesmo perfil geral e a v2 ficou de 4,28% a 6,74% acima da v1 em energia. Isso é um bom exemplo de que uma tentativa de otimização pode não funcionar para a distribuição de entradas usada no experimento. O aprendizado aqui não é que a v2 seja universalmente pior, mas que seu benefício não apareceu nesta carga.

Insertion Sort: foi onde uma melhoria de implementação teve maior efeito sem exigir a mudança para uma classe assintótica diferente. A energia caiu 98,10% no tamanho de 50.000. O ganho acompanha diretamente a redução do tempo, de 35,762 s para 0,839 s.

Selection Sort: é o caso clássico em que a escolha do algoritmo domina o resultado. A versão 1 apresentou comportamento quadrático, enquanto a versão 2, baseada em HeapSort, cai para $O(n \log n)$. Em 50.000 elementos, a energia caiu 99,58% e o tempo caiu 99,42%.

11. Limitações e cuidados na interpretação

Os resultados são fortes, mas não devem ser tratados como uma lei universal. Primeiro, as conclusões dependem do hardware usado. Um processador diferente, outro sistema operacional, outra versão do Python ou outra linguagem de programação pode mudar as constantes e até a ordem prática entre versões. Segundo, a distribuição dos dados de entrada importa. Uma melhoria do Bubble Sort, por exemplo, pode se comportar muito melhor em dados já ordenados ou quase ordenados do que em entradas aleatórias.

Terceiro, as medições de energia são sensíveis a ruído externo. Por isso, trabalhos metodologicamente mais rigorosos recomendam aquecimento da máquina, redução de tarefas em segundo plano, repetição e análise da distribuição [2]. No caso deste trabalho, não foi possível monitorar o aquecimento das máquinas utilizadas, foram utilizadas duas máquinas diferentes com a mesma configuração no mesmo laboratório e nas mesmas condições, foi reduzido o máximo possível as tarefas em segundo plano, rodando os códigos apenas com a pasta dos arquivos e o terminal onde rodava o código. As duas máquinas rodavam as mesmas versões de Python, o mesmo sistema operacional e o mesmo código.

12. Conclusão: eficiência de software também é uma questão de energia

O experimento mostra de maneira concreta que escolhas de implementação podem ter impacto físico. Não se trata apenas de produzir uma saída correta; duas soluções que ordenam exatamente o mesmo conjunto de dados podem exigir quantidades muito diferentes de processamento e, portanto, de energia.

Os resultados mais significativos apareceram quando a implementação mudou o perfil de execução. A versão 2 do Selection, com HeapSort e complexidade $O(n \log n)$, reduziu quase toda a energia associada à versão quadrática para o conjunto de 50.000 elementos. O Insertion v2 também obteve enorme redução, mostrando como diminuir movimentos e operações internas pode fazer diferença mesmo dentro da mesma classe assintótica. O Bubble v2, por sua vez, serve como contraponto: nem toda modificação chamada de otimização resulta em economia para todas as cargas.

A principal mensagem do estudo é simples: complexidade, implementação e hardware precisam ser analisados em conjunto. O tempo de execução ajuda a explicar grande parte da energia medida, mas detalhes como memória, cache e padrão de acesso também importam. Medir essas diferenças transforma uma discussão teórica sobre algoritmos em uma observação concreta do custo computacional.

Referências e modelo consultado

- [1] SustainableSE. Energy Consumption of Sorting Algorithms: Impact of Algorithmic Complexity and Memory Behavior. Group 19, 2026.
https://luiscruz.github.io/course_sustainableSE/2026/p1_measuring_software/g19_SortingAlgorithms.html
- [2] CRUZ, Luís. Green Software Engineering Done Right: a Scientific Guide to Set Up Energy Efficiency Experiments. 2021. <https://luiscruz.github.io/2021/10/10/scientific-guide.html>
- [3] Linux Manual Pages. perf-stat(1) — Run a command and gather performance counter statistics.
<https://www.man7.org/linux/man-pages/man1/perf-stat.1.html>
- [4] Linux Manual Pages. perf-list(1) — Tool events such as duration_time, user_time and system_time.
<https://www.man7.org/linux/man-pages/man1/perf-list.1.html>

Fonte dos resultados quantitativos: planilha do experimento enviada para esta análise; os valores apresentados no texto são médias das dez execuções por combinação.